

Engineering exercise

Build a posture sensor

We give you a real bare metal server for 10 hours. Write something that inspects it and reports what it finds, in a shape we can read.

About 4 hours of work

Any language, Go preferred

Real bare metal, not a VM

Claude required

What we are looking at

01

Judgment

No sensor can answer every question about every machine, and that is by design. We care less about covering every edge case, and more about how gracefully your sensor handles unanswerable checks while maintaining a clean, generic architecture.

02

How you work with Claude

We use it every day and you should too. What we read is how you direct it: what you specified before generating, what you rejected, and what you verified before believing it.

03

Safe on someone's machine

This kind of tool runs on hardware other people depend on. Read only, bounded in time and output, and it does not fall over on something it did not expect to see.

What We Are Not Testing

- **Prior knowledge of server hardware:** BMCs, IPMI, firmware. If these are unfamiliar, that is completely fine. Everything you need is in this brief or a quick search away, and AI tools (like Claude) know this domain well.
- **Go fluency:** Write in whatever language makes you fastest. We ship in Go, but your language choice will not affect your score.
- **Volume:** A handful of checks done thoughtfully beats twenty done thinly.

What you get from us

01

A bare metal server

An SSH key and an address, yours for a 10 hour window you pick. Real hardware in a real data center, not a container or a VM pretending to be one, so what you read off it is true.

02

An Anthropic API key

On our account rather than yours, so you are not spending your own credit. It is live for your window and we retire it afterwards.

03

The output shape

This brief, plus `finding.schema.json`, which your output has to validate against.

Work wherever you prefer. Write the code on your own laptop and run it against the server over SSH, or work directly on the server itself with Claude installed there. Both are completely fine and neither is scored. The cheatsheet below has the commands for either route.

Cheatsheet

So you do not lose twenty minutes to plumbing. Replace `KEY` with the path to the key we sent you, and `USER` and `HOST` with the username and address from the same email.

Connect

```
ssh -i KEY USER@HOST
```

Copy your code up to the server

```
# a whole directory, repeatable, skips unchanged files
rsync -avz -e "ssh -i KEY" ./sensor/ USER@HOST:~/sensor/

# or a one off file
scp -i KEY ./sensor.py USER@HOST:~/
```

Copy results back down to your laptop

```
scp -i KEY USER@HOST:~/findings.json .

# or a whole directory
rsync -avz -e "ssh -i KEY" USER@HOST:~/sensor/ ./sensor/
```

Run one command without logging in

```
ssh -i KEY USER@HOST 'cat /proc/cmdline'
```

Install and configure Claude Code on the server

```
curl -fsSL https://claude.ai/install.sh | bash && \  
  echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc && \  
  echo 'export ANTHROPIC_API_KEY="sk-ant-REPLACE_ME"' >> ~/.bashrc && \  
  source ~/.bashrc && claude --version
```

The task

Write a command line tool that inspects the server and writes what it found to a file.

```
sensor scan --out findings.json
```

Part one: describe the machine

Before any of the security work, the output has to say what this machine actually is. It is the least glamorous part and it matters more than it looks: a finding with no machine attached to it is not worth much once you have more than one server.

Report at least these, and add whatever else you think belongs:

- **Identity.** A host identifier that stays the same across runs, and the hostname.
- **Owner.** Who this machine belongs to, as far as the machine itself can tell.
- **Hardware.** Vendor and model, CPU model and core count, memory.
- **Operating system.** Distribution, version, kernel release.
- **Storage.** The block devices attached, their models and sizes.

If you cannot determine one of these, say so in the output. An empty string or a plausible guess is worse than a field that admits it does not know.

Part two: check its posture

Organize your checks within the provided **categories**. Each category represents a logical domain and should contain at least two relevant checks (or as many as needed), with each check yielding its own distinct result.

So a category with four checks in it produces four findings, each with its own status and severity. Severity **info** is a real answer, not a cop out. Plenty of things are worth reporting because an operator should know them, not because they are broken.

Three categories we ask for

We describe each one by what it should answer. Working out what that means on this particular machine, and how to establish it rather than assume it, is the exercise.

Category	What it should answer
REMOTE_ACCESS	How this machine can be logged into from elsewhere, and what that permits. Who may connect, by what means of authentication, and as whom. The interesting part is what is actually in force on the running system, which is not always what any single configuration file says.
SECRETS_ON_DISK	What credential material is sitting on this machine, and who can read it. Keys, tokens, passwords, certificates, provisioning data, anything someone could pick up and reuse somewhere else. Both what is there and how well it is protected.
BMC_INBAND_ACCESS	Whether this machine can talk to its own baseboard management controller from inside the operating system, through which interface, and who is permitted to do it.

And at least one category of your own

Look around the machine and pick another subject worth reporting on for a server sitting in someone's data center. One line in `NOTES.md` on why you chose it. More than one is welcome.

As a benchmark, **Kernel Flags** makes a solid category. A single audit here can cover whether hardware security features are active, if unsigned code is permitted, or if the boot chain was verified, each requiring its own check.

Treat this as a baseline. There is no predefined solution set here, so be creative. Think about a typical, forgotten production server and the hidden drift it accumulates. What impresses us most is discovering a category we did not anticipate, supported by clear engineering intent.

Some context on the BMC, since it is the least familiar one

Every server has a baseboard management controller: a small independent computer on the motherboard that stays powered while the host is off. It can power cycle the machine, mount virtual media, and reflash firmware, which makes it the most privileged thing in the chassis. It normally answers on its own dedicated network port, which you do not have access to here.

A host may also be able to reach it from the inside, over an interface called KCS, without touching the network at all. That path is not always present and not always permitted, which varies from machine to machine. Where it exists, Linux exposes it through the `ipmi_si` and `ipmi_devintf` drivers and a device node, and `ipmitool` is the usual client for it. Whether it exists on this machine, and who is allowed to walk it, is the question.

The output shape

Your `findings.json` has to validate against this. We fix the shape because a stable one is what makes a finding usable by anything other than a person reading a terminal, and because it lets us compare submissions fairly.

```
{
  "schema_version": "1",
  "collected_at": "<RFC 3339 timestamp>",
  "sensor_version": "<optional>",

  "machine": {
    "host_id": "<stable across runs>",
    "hostname": "...",
    "owner": "<or an explicit unknown>",
    "vendor": "...",
    "model": "...",
    "os": { "name": "...", "version": "...", "kernel": "..." },
    "cpu": { "model": "...", "cores": 0 },
    "memory_bytes": 0,
    "storage": [ { "device": "...", "model": "...", "size_bytes": 0 } ]
  },

  "findings": [
    {
      "category": "REMOTE_ACCESS",          // UPPER_SNAKE_CASE
      "check_id": "ROOT_LOGIN_PERMITTED",  // unique per check
      "status": "pass" | "fail" | "unknown",
      "severity": "critical" | "high" | "medium" | "low" | "info",
      "title": "<one line a person can read>",
      "reason": "<required when status is fail or unknown>",
      "evidence": { /* what someone needs to act on it */ },
      "collected_at": "<RFC 3339 timestamp>"
    }
  ]
}
```

Extra fields are welcome anywhere, so put real evidence in there rather than a restatement of the title. A good `evidence` block means the reader does not have to run your tool again to believe you: the path you read, the value you found, the command you ran, the errno you got back.

Rules

Required

- **Read only.** The sensor observes the machine, it never changes it.
- **Bounded.** Every subprocess gets a timeout, output stays a sane size, and nothing hangs forever on a device that does not answer.
- **It does not crash.** One check hitting something unexpected should not take the other checks down with it.
- **Unprivileged.** It runs as the user we give you, with no escalation.
- **Use Claude,** and send us the session.

Explicitly out of scope

None of this is graded, and reaching for it is the most common way a four hour exercise becomes a ten hour one.

No daemon or scheduler

No database

No UI

No API server

No packaging

No CI

The part people get wrong

A sensor cannot always assess every state of a machine. When a check is unexecutable or out of reach, **state that explicitly**. A report that silently omits unreached checks, pretending a scan was thorough, destroys trust in monitoring tools. Returning no issues is a valid outcome; claiming to have checked when you could not is a critical failure.

The Follow-Up Conversation

Once we have read your submission, we will sit down together. There is nothing to prepare and nothing extra to submit for this, but it is only fair that you know what it covers.

Your Sensor and Findings: We will take a high-level look at your code and findings to discuss your approach, key design choices, and what you would tweak after seeing it run live.

How This Becomes a Product: Yours runs on a single reachable machine; ours runs on tens of thousands in isolated customer data centers, reporting to a central plane.

A few topics we will dive into, just so you know what to expect:

- **No way in, only a way out:** The sensor sits in a data center you cannot reach. It can open connections outward, nothing can open one towards it. How does configuration get to it, and how do you ship it a new version?
- **Which customer is this?** A sensor starts up for the very first time and says hello. How do you recognize which customer it belongs to, before anyone trusts a single finding from it?
- **Volume:** Ten thousand machines, five hundred findings each, every scan. What crosses the network and how often, and what do you keep once it lands? Every scan, or only what changed?
- **Isolation:** Where is the boundary between two customers' data, and what makes a cross-customer read impossible rather than unlikely?

What to send us

One tarball, by email. No repository to create and no accounts to link.

01 The source

Plus the one command that runs it. If we cannot run it, we cannot give you credit for it.

02 findings.json

From a real run against the server we gave you, not a hand-written example.

03 NOTES.md, one page at most

What you assumed, what turned out to be ambiguous and how you settled it. The category you added and why. What you would do with another day. And one thing Claude got wrong that you rejected, with the reason.

04 Your Claude session

The transcript, as exported. We read these, and a messy real one is far better received than a tidied one.

Practicalities

The four hours	A guide, not a stopwatch. We are not timing you and nobody is comparing timestamps.
Questions	Ask us, at any point. Working out loud is not cheating here, and needing no clarification at all is not a score. If something in this brief is unclear, that is usually our bug rather than yours.
If the server breaks	Tell us and we will fix it. It will not count against you and it will not eat your window.